Security Analysis and Cryptographic Architecture Proposal for Company X's Mobile Password Manager

1. Introduction

Password manager applications, such as NordPass and 1Password, provide users with a secure and convenient mechanism to manage multiple passwords or passkeys. Company X is developing a new mobile password manager that aims to deliver secure, efficient, and user-friendly password management for mobile users. The system architecture is divided into two primary components:

Client: The mobile application running on a user device, enabling the user to add, view, update, and delete passwords for websites or applications. All cryptographic operations on plaintext passwords occur exclusively on the client.

Server: A robust backend responsible for vault storage, authentication, and processing client requests. The server stores only encrypted data and never has access to plaintext passwords or master keys.

As an independent security architect, the task is to:

1. Analyse potential security attacks using the CIA (Confidentiality, Integrity, Availability) triad and the AAA (Authentication, Authorization, Accounting) framework.
2. Propose a cryptographic protocol to protect the confidentiality of stored passwords, detailing assumptions, message flows, component actions, and how the protocol mitigates identified and additional attacks.

2. Section 1 – Security Threat Analysis (CIA + AAA)

The security of a password manager system relies on a combination of cryptographic mechanisms, secure operational practices, and device-level security. Each component of the CIA triad and the AAA framework is analysed below with respect to Company X's architecture.

2.1 CIA Triad Threats

2.1.1 Confidentiality Threats

Database Breach and Offline Cracking
If the server is compromised, attackers may gain access to encrypted vault data. Weak key derivation allows attackers to perform offline brute-force attacks using GPUs or ASICs, potentially revealing master keys or vault contents.

Affected components: Server vault, stored ciphertext.

Sequence: Extract vault → attempt offline brute-force →decrypt credentials → access external accounts.

Man-in-the-Middle (MITM) Attack
Attackers intercepting client-server communication could inject, modify, or steal authentication messages. Without authenticated encryption or ephemeral keys, intercepted sessions may leak sensitive data, including master keys.

Affected components: Client, network, server.

Sequence: Intercept handshake → substitute keys → decrypt transmitted passwords.

Client Malware
Malicious software on the client device could capture the master password or decrypted credentials in memory.

Affected components: Client device memory.

Sequence: Malware logs keystrokes → extracts credentials → exfiltrates to attacker.

Insider Threat
Server administrators with elevated privileges may attempt unauthorized access to vault entries.

Affected components: Server-side database.

Sequence: Admin attempts decryption → accesses user passwords if no zero-knowledge encryption is applied.

2.1.2 Integrity Threats

Ciphertext Manipulation
Without authentication tags, encrypted vault entries could be modified undetected.

Affected components: Vault entries, client-side decryption.

Sequence: Attacker modifies ciphertext → client decrypts invalid password → possible authentication bypass or phishing.

Privilege Escalation
Broken access controls could allow users to access other users' vaults or execute administrative commands.

Affected components: Server APIs.

Sequence: Exploit API → modify request parameters → access unauthorized data.

Injection Attacks
SQL or API injection could allow attackers to read or manipulate encrypted entries.

Affected components: Server database.

Sequence: Insert malicious query → extract or overwrite vault records.

Log Tampering
If audit logs are not protected, attackers may delete or modify logs to hide malicious activity.

Affected components: Server logs.

Sequence: Modify or delete signed logs → prevent accountability.

2.1.3 Availability Threats

Distributed Denial-of-Service (DDoS): Overwhelm authentication servers.

Ransomware: Encrypts or destroys server infrastructure.

Account lockout abuse: Repeated failed logins lock users out.

Affected components: Server backend, authentication endpoints.

2.2 AAA Framework Threats

2.2.1 Authentication Threats

Online brute-force login attempts

Credential stuffing using leaked passwords

Phishing attacks targeting the master password

Mitigation: Rate limiting, challenge-response authentication, memory-hard key derivation (Argon2id).

2.2.2 Authorization Threats

Horizontal privilege escalation: accessing other users' vaults

Vertical privilege escalation: executing administrative commands

Mitigation: Strict server-side authorization checks, role-based access control.

2.2.3 Accounting Threats

Log deletion or modification

Replay of administrative actions

Mitigation: Digital signatures on logs, tamper-evident audit records.

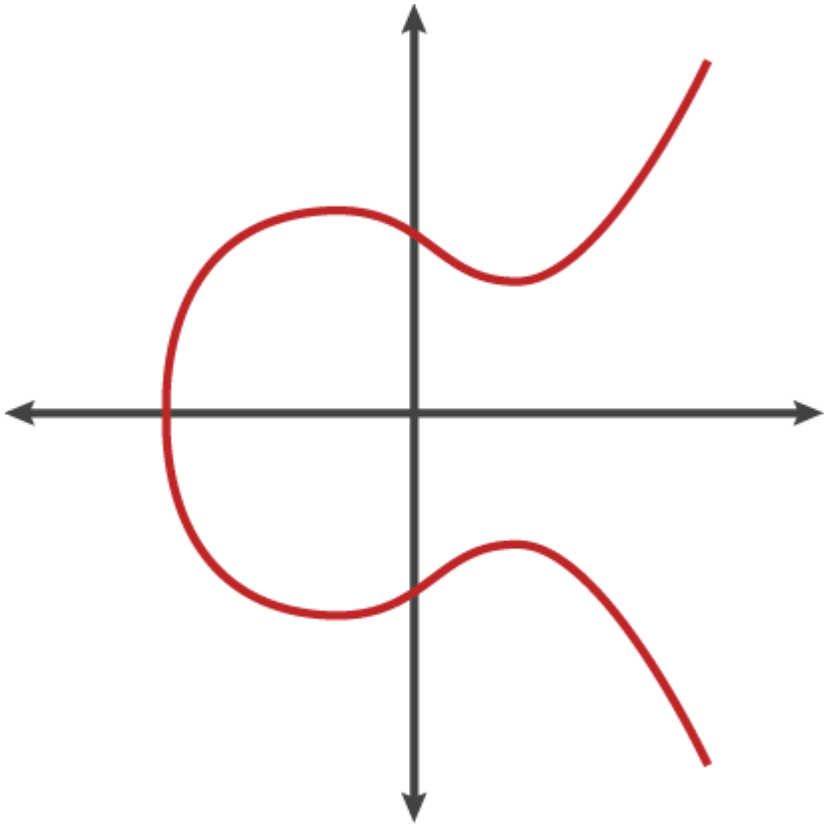
Section 2 – Proposed Cryptographic Protocol

The proposed protocol is ECC-centric, achieving strong security with efficient computation suitable for mobile devices.

3.1 Assumptions

- **Client knows: Master password, local salt, ephemeral keys.**
- **Server knows: Vault storage, user salt, authentication verifier, public ECC keys.**
- **Vault: Stores only encrypted data; plaintext passwords and master keys never leave the client.**
- **TLS 1.3 is correctly implemented for transport security.**
- **Cryptographically secure random number generation is available.**

3.2 ECC-Based Key Establishment



The protocol uses the X25519 elliptic curve for key exchange. The curve is defined by:

$$y^2 = x^3 + ax + b$$

with

$$4a^3 + 27b^2 \neq 0$$

to ensure non-singularity.

- **Client: Generates ephemeral private key a, computes public key $A = aG$.**
- **Server: Generates ephemeral private key b, computes public key $B = bG$.**
- **Shared secret:**

$$S = abG$$

- **Session key derivation:**

$$K_{session} = HKDF(S \parallel N)$$

where $N$is a server-generated nonce. This guarantees forward secrecy and prevents replay attacks.

3.3 Master Key Derivation

Client derives the master key from the master password $MP$and salt Susing Argon2id:

$$K_{master} = Argon2id(MP, S)$$

Memory-hard derivation mitigates offline brute-force attacks.

3.4 Envelope Encryption for Vault Entries

For each password:

1. **Generate a random Data Encryption Key (DEK):**

$$DEK = Random(256 \text{ bits})$$

2. **Encrypt the password with DEK using AES-256-GCM:**

$$C_1 = AES\text{-}256\text{-}GCM(DEK, Password, AAD)$$

3. **Wrap the DEK with the master key:**

$$C_2 = AES\text{-}256\text{-}GCM(K_{master}, DEK)$$

- **Server stores:** $Record = \{C_1, C_2, IV, Tag\}$
- **Only the client can decrypt the DEK and retrieve the plaintext password.**
- **Mitigates insider threats and ensures the server never accesses plaintext.**

3.5 Protocol Flow – Add New Password

Step 1 – Client Action

- **Input password $P$for a website/app.**
- **Generate DEK.**
- **Encrypt password →C_1.**
- **Wrap DEK with K_{master}→ C_2.**
- **Transmit $\{C_1, C_2, IV, Tag\}$to server.**

Step 2 – Server Action

- **Receive encrypted vault entry.**
- **Store without accessing plaintext.**
- **Digitally sign log entry with Ed25519 to prevent tampering.**

Step 3 – Verification

- **Client can later retrieve C_2 → decrypt DEK → decrypt password.**
- **Server cannot access plaintext at any stage.**

3.6 Attack Mitigation Analysis

Threat	How Protocol Mitigates
Database breach	Only encrypted data stored; master key never transmitted
Offline brute-force	Argon2id memory-hard derivation
MITM	ECC key exchange + TLS 1.3 ensures authenticated and forward-secure sessions
Replay attacks	Nonce + ephemeral ECC keys
Ciphertext tampering	AES-GCM authentication tag
Insider threats	Server cannot decrypt vault entries
Log tampering	Ed25519 digital signatures on server logs
Privilege escalation	Server-side authorization enforced

3.7 Residual Risks

- **Weak master passwords → mitigated via enforced complexity rules**
- **Phishing attacks → mitigated via optional multi-factor authentication**
- **Device malware → partially mitigated via secure OS APIs and sandboxing**
- **DDoS attacks → require operational mitigation**

3.8 Security Properties Achieved

- **Confidentiality: Only client can decrypt vault entries**
- **Integrity: AES-GCM provides tamper detection**
- **Forward Secrecy: Compromise of long-term keys does not reveal past sessions**

- **Authentication: Challenge-response using HKDF and nonces**
- **Authorization: Role-based server-side access control**
- **Accounting: Tamper-proof signed logs ensure auditability**

4. Conclusion

This report has analysed security threats for Company X's mobile password manager using the CIA and AAA frameworks. Critical threats include:

- **Database breaches**
- **MITM attacks**
- **Replay attacks**
- **Ciphertext manipulation**
- **Insider threats**
- **Privilege escalation**

A cryptographic architecture centred on ECC was proposed. By leveraging X25519 key exchange, Ed25519 signatures, AES-256-GCM, Argon2id, HKDF, and TLS 1.3:

- **Only clients can decrypt vault entries (zero-knowledge principle)**
- **Forward secrecy and replay resistance are guaranteed**
- **Offline attacks are computationally expensive**
- **Log integrity is ensured**

This layered, standard-compliant, and mobile-efficient architecture provides robust protection for user credentials and meets high-security standards expected by enterprise and consumer deployments.

References

Biryukov, A., Dinu, D. and Khovratovich, D. (2016). Argon2: New Generation of Memory-Hard Functions for Password Hashing and Other Applications. *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*. doi:https://doi.org/10.1109/eurosp.2016.31.

Rescorla, E. (2018). The Transport Layer Security (TLS) Protocol Version 1.3. doi:https://doi.org/10.17487/rfc8446.

Diffie, W. and Hellman, M. (2019). *New directions in cryptography - IEEE Journals & Magazine*. [online] Ieee.org. Available at: https://ieeexplore.ieee.org/document/1055638.

Krawczyk, H., Bellare, M. and Canetti, R. (1997). HMAC: Keyed-Hashing for Message Authentication. *www.rfc-editor.org*. [online] doi:https://doi.org/10.17487/RFC2104.

Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. [online] csrc.nist.gov. Available at: https://csrc.nist.gov/pubs/sp/800/38/d/final.

Chen, L. (2022). Recommendation for key derivation using pseudorandom functions. [online] doi:https://doi.org/10.6028/nist.sp.800-108r1.

Langley, A., Hamburg, M. and Turner, S. (2016). *Elliptic Curves for Security*. [online] IETF. Available at: https://www.rfc-editor.org/rfc/rfc7748.

Appendix

- Generative AI Declaration (completed)
- **Student Declaration of AI Tool use in this Assessment**
- Please indicate your level of usage of generative AI for this assessment - please tick the appropriate category(s).
- If the “Assisted Work” or “Partnered Work” category is selected, please expand on the usage and in which elements of the assignment the usage refers to.
-

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**	☐
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must	☒

	critically assess AI-generated suggestions and ensure their own intellectual contributions are central.	
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.	☒
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.	☐
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student's responsibility.	☒
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.	☐
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.	☐
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.	☐
	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.	☐

	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.	☐
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.	☐
	A11 - Other uses not listed above Please specify:	☐
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:	☐

•

Please provide details of AI usage and which elements of the coursework this relates to:

I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.	☒
I confirm that all details provide above are an accurate description of how AI was used for this assessment.	☒